

Unit 2: Central Processing Unit (CPU)

- *Structure and function of CPU*
 - *Arithmetic Logic Unit (ALU): Design and operation*
 - *Control Unit (CU): Hardwired vs. Microprogrammed*
 - *Instruction cycle and execution process*
 - *Types of instructions: Data transfer, Arithmetic, Logical, Control flow*
 - *Instruction formats and addressing modes*
 - *Register organization: General-purpose and special-purpose registers*
 - *Program counter, stack pointer, and status register*
 - *Interrupt handling and exception processing*
 - *Introduction to Pipelining and its types (Instruction and Arithmetic)*
-

Structure and Function of CPU

Structure of CPU

The Central Processing Unit (CPU) is the core component of a computer responsible for executing instructions. It is composed of the following parts:

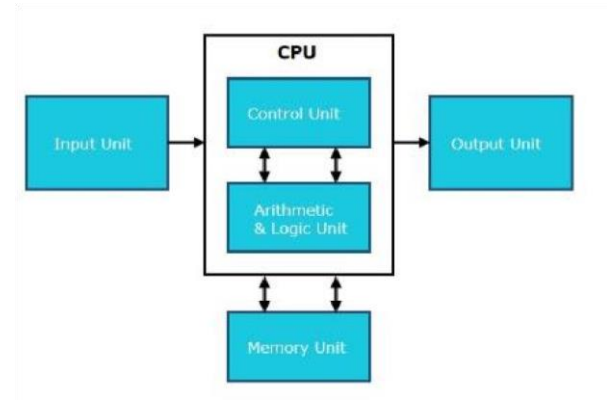
- 1. Arithmetic Logic Unit (ALU): Handles arithmetic and logical operations.*
- 2. Control Unit (CU): Directs the operation of the processor by coordinating activities between the CPU and other components.*
- 3. Registers: Temporary storage locations used for fast data access during processing.*
- 4. Cache Memory: High-speed memory storing frequently accessed data and instructions.*

5. *Clock: Synchronizes the operations of the CPU.*

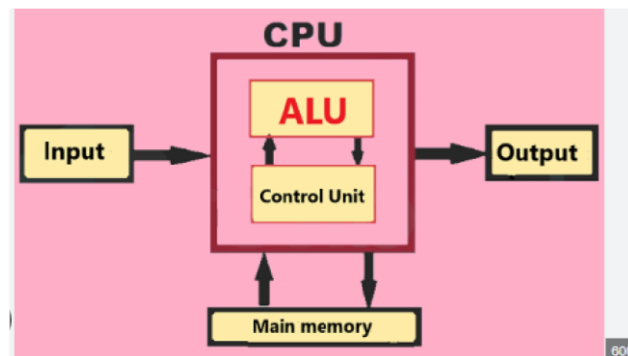
Function of CPU

The CPU performs four main functions:

1. *Fetch: Retrieves instructions from memory.*
2. *Decode: Deciphers the fetched instructions.*
3. *Execute: Carries out the operations specified by the instructions.*
4. *Store: Saves the results of operations in memory or registers.*



Arithmetic Logic Unit (ALU): Design and Operation



Design of ALU

- *The ALU is a combinational circuit composed of arithmetic and logical units.*
- *Main components:*
 - *Adders/Subtractors: Perform addition and subtraction.*
 - *Logic Gates: Handle logical operations*

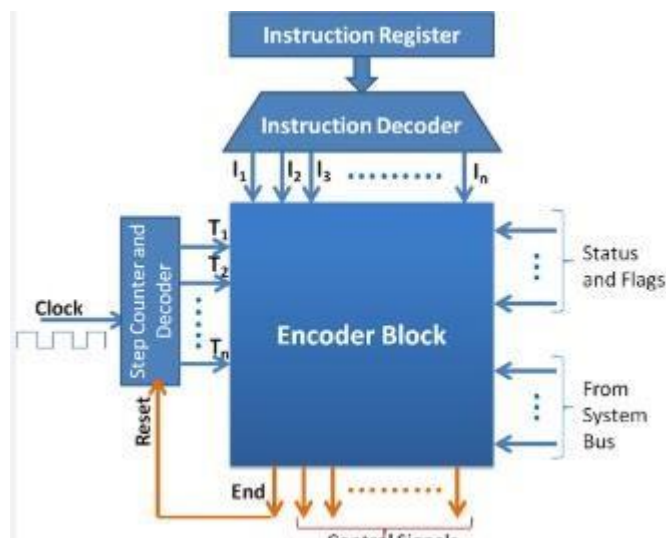
(AND, OR, NOT, XOR). ○ Multiplexers: Select specific operations based on control signals.

Operation of ALU

1. *Input: Data inputs are provided by registers or memory.*
2. *Control Signals: Determine the operation to be performed.*
3. *Output: Result is sent back to a register or memory.*

Control Unit (CU): Hardwired vs. Microprogrammed

Control Unit



The Control Unit generates control signals to manage the execution of instructions.

Types of Control Units

1. *Hardwired Control Unit:*
 - *Uses fixed combinational logic circuits to generate control signals.*
 - *High speed but inflexible.*
2. *Microprogrammed Control Unit:*

- Uses a set of microinstructions stored in control memory to generate control signals.
- Flexible but slower than hardwired units.

Differences between Hardwired and Microprogrammed CU

Difference between Hardwired and Microprogrammed Control Unit:

ATTRIBUTES	HARDWIRED CONTROL UNIT	MICROPROGRAMMED CONTROL UNIT
1. Speed	Speed is fast	Speed is slow
2. Cost of Implementation	More costly.	Cheaper.
3. Flexibility	Not flexible to accommodate new system specification or new instruction redesign is required.	More flexible to accommodate new system specification or new instruction sets.
4. Ability to Handle Complex Instructions	Difficult to handle complex instruction sets.	Easier to handle complex instruction sets.
5. Decoding	Complex decoding and sequencing logic.	Easier decoding and sequencing logic.
6. Applications	RISC Microprocessor	CISC Microprocessor
7. Instruction set of Size	Small	Large
8. Control Memory	Absent	Present
9. Chip Area Required	Less	More
10. Occurrence	Occurrence of error is more	Occurrence of error is less

<i>Feature</i>	<i>Hardwired CU</i>	<i>Microprogrammed CU</i>
<i>Design Complexity</i>	<i>Complex</i>	<i>Simpler</i>
<i>Flexibility</i>	<i>Limited</i>	<i>High</i>
<i>Speed</i>	<i>Faster</i>	<i>Slower</i>
<i>Implementation</i>	<i>Hardware-based</i>	<i>Firmware-based</i>

<i>Modification</i>	<i>Difficult</i>	<i>Easy</i>
<i>Cost</i>	<i>High</i>	<i>Relatively lower</i>
<i>Usage</i>	<i>Simple CPUs</i>	<i>Complex CPUs</i>
<i>Debugging</i>	<i>Difficult</i>	<i>Easier</i>
<i>Scalability</i>	<i>Poor</i>	<i>Better</i>
<i>Example</i>	<i>Older processors</i>	<i>Modern processors</i>

Instruction Cycle and Execution Process

Instruction Cycle

Computer Instructions

Computer instructions are a set of machine language instructions that a particular processor understands and executes. A computer performs tasks on the basis of the instruction provided.

An instruction comprises of groups called fields. These fields include:

- The Operation code (Opcode) field which specifies the operation to be performed.*
- The Address field which contains the location of the operand, i.e., register or memory location.*
- The Mode field which specifies how the operand will be located.*



The Instruction Cycle is the sequence of steps the CPU takes to execute an instruction. It includes the following stages:

- 1. Fetch: Retrieve the instruction from memory using the Program Counter (PC).*
- 2. Decode: Interpret the instruction to identify the operation and operands.*

3. *Execute: Perform the operation specified (e.g., arithmetic or data transfer).*
4. *Write Back: Store the result in a register or memory.*

Key Registers Involved

- *Program Counter (PC): Points to the next instruction.*
- *Instruction Register (IR): Holds the current instruction.*
- *Memory Address Register (MAR): Holds the address of the instruction/data.*
- *Memory Data Register (MDR): Temporarily holds data to/from memory.*

Types of Instructions

1. *Data Transfer Instructions:*
 - *Move data between registers, memory, or I/O devices.*
 - *Examples: MOV, LOAD, STORE.*
2. *Arithmetic Instructions:*
 - *Perform arithmetic operations.*
 - *Examples: ADD, SUB, MUL, DIV.*
3. *Logical Instructions:*
 - *Perform logical operations.*
 - *Examples: AND, OR, XOR, NOT.*
4. *Control Flow Instructions:*
 - *Alter the sequence of instruction execution.* ◦ *Examples: JMP, CALL, RET, BRANCH.*

Instruction Formats and Addressing Modes

Instruction Formats

Instruction formats define the structure of an instruction. Components include:

- *Opcode: Specifies the operation to be performed.*
- *Operands: Indicate data or memory addresses.*

Types of Instruction Formats

1. *Zero Address: No explicit operands (e.g., stack operations).*
2. *One Address: One operand (e.g., accumulator-based).*
3. *Two Address: Two operands (e.g., MOV A, B).*
4. *Three Address: Three operands (e.g., ADD A, B, C).*

Addressing Modes

Addressing modes define how operands are accessed in memory or registers:

1. *Immediate: Operand is part of the instruction.*
2. *Register: Operand is stored in a register.*
3. *Direct: Address of the operand is provided.*
4. *Indirect: Address of the operand is held in a register.*
5. *Indexed: Combines a base address and an offset.*

Register Organization

Types of Registers

1. *General-Purpose Registers:*
 - *Used for temporary data storage.*
 - *Examples: R0, R1, R2, etc.*
2. *Special-Purpose Registers:*
 - *Perform specific functions. ◦ Examples: Program Counter (PC), Stack Pointer (SP), Status Register (SR).*

Key Special-Purpose Registers

- *Program Counter (PC): Tracks the address of the next instruction.*
- *Stack Pointer (SP): Points to the top of the stack.*
- *Status Register (SR): Holds flags indicating CPU states (e.g., Zero, Carry).*

Interrupt Handling and Exception Processing

Interrupt Handling

Interrupts are signals that temporarily halt CPU operations to handle external or internal events.

Steps in Interrupt Handling

- 1. Save the current CPU state.*
- 2. Jump to the Interrupt Service Routine (ISR).*
- 3. Execute the ISR.*
- 4. Restore the CPU state and resume operations.*

Exception Processing

Exceptions are special conditions (e.g., divide by zero, invalid memory access) that require immediate handling. Types of Exceptions

- 1. Synchronous Exceptions: Occur during instruction execution (e.g., arithmetic errors).*
 - 2. Asynchronous Exceptions: External events like hardware failures.*
-

Introduction to Pipelining and Its Types

Pipelining in CPUs

Pipelining is a technique used in CPU design to improve performance by allowing multiple instructions to overlap during execution. Instead of executing one instruction at a time from start to finish, pipelining divides the instruction execution process into distinct stages, allowing the CPU to work on several instructions simultaneously.

Stages of a Pipeline

A typical instruction pipeline consists of the following stages:

1. *Fetch (F):* ○ The instruction is fetched from memory.
2. *Decode (D):* ○ The fetched instruction is decoded to determine the operation and operands.
3. *Execute (E):*
 - The operation is performed (e.g., addition, multiplication).
4. *Memory Access (M):* ○ Data is read from or written to memory, if required.
5. *Write Back (WB):*
 - The result of the operation is written back to the register.

Pipelining is a technique to improve CPU performance by overlapping instruction execution stages.

Types of Pipelining

1. *Instruction Pipelining:* Divides the instruction cycle into stages (e.g., Fetch, Decode, Execute) to execute multiple instructions simultaneously.
2. *Arithmetic Pipelining:* Breaks arithmetic operations into smaller steps performed in parallel.

Advantages of Pipelining

- *Increases instruction throughput.*
- *Efficient use of CPU resources.*

Challenges of Pipelining

- *Data Hazards:* Dependencies between instructions.
- *Control Hazards:* Issues with branch instructions.
- *Structural Hazards:* Resource conflicts.

General-Purpose Registers (GPRs):

General-purpose registers are versatile and can store various types of data or perform different operations. They are used for temporary data storage during program execution.

Key Characteristics:

1. *Multi-functional:*
 - *They are not restricted to a single type of operation.*
 - *Used for arithmetic operations, logical operations, and data movement.*
2. *Speed:*
 - *Extremely fast since they are located inside the CPU.*
 - *Used to reduce the overhead of accessing slower memory units (like RAM or cache).*
3. *Examples (Architecture-specific):*
 - *x86 Architecture (Intel/AMD):*
 - *EAX, EBX, ECX, EDX (32-bit registers in older processors).*
 - *RAX, RBX, RCX, RDX (64-bit versions in modern processors).*
 - *ARM Architecture:*
 - *Registers R0-R15 are general-purpose, with specific registers sometimes reserved for certain tasks (e.g., R13 for the stack pointer).*
4. *Common Uses:* ○ *Temporary storage for intermediate values during execution.* ○ *Loop counters (e.g., in for loops).* ○ *Operand storage for arithmetic instructions.*

Special-Purpose Registers (SPRs):

Special-purpose registers are dedicated to specific roles in the CPU. They handle specialized tasks related to the control and operation of the processor.

Key Characteristics:

1. *Single-purpose:*
 - *These registers are designed for specific tasks and cannot be used for general operations.*
2. *Critical for Processor Control:*
 - *Manage program execution, CPU status, and memory addressing.*
3. *Examples (Architecture-specific):*

- *Instruction Pointer (IP) or Program Counter (PC):*
 - *Holds the address of the next instruction to be executed.*
 - *Stack Pointer (SP):*
 - *Points to the top of the stack in memory.*
 - *Status Register (Flags Register):*
 - *Stores flags that indicate the outcome of operations (e.g., zero flag, carry flag).*
 - *Control Registers (CR):*
 - *Found in x86 processors, used for memory management and system control.*
 - *Base/Index Registers:*
 - *Used in addressing modes for memory access (e.g., in x86: EBP/ESP).*
4. *Other Examples:*
- *Segment Registers (in x86): Used for segmented memory management.*
 - *Interrupt Descriptor Table Register (IDTR): Points to the interrupt descriptor table.*
 - *Timer Registers: Control hardware timing and clock events.*

Comparison between GPRs and SPRs

<i>Aspect</i>	<i>General-Purpose Registers (GPRs)</i>	<i>Special-Purpose Registers (SPRs)</i>
<i>Functionality</i>	<i>Multi-purpose; stores data, operands, or intermediate results.</i>	<i>Dedicated to specific tasks (e.g., program execution, control).</i>
<i>Flexibility</i>	<i>High – can be used in most operations.</i>	<i>Low – restricted to specialized tasks.</i>
<i>Examples</i>	<i>EAX, EBX, R0-R15</i>	<i>PC, SP, Flags, Control Registers</i>
<i>Role in Execution</i>	<i>Performs core computation and temporary data storage.</i>	<i>Maintains the state and flow of the program.</i>

Role of Program Counter (PC)

The Program Counter (PC) is a special-purpose register in the CPU that:

1. *Holds the Address of the Next Instruction:*
 - *The PC contains the memory address of the next instruction to be fetched and executed.*
2. *Updates Automatically:*
 - *After fetching an instruction, the PC is incremented (in sequential execution) or updated with a new address in case of a jump or branch.*
3. *Ensures Sequential Execution:*
 - *It maintains the logical flow of a program by pointing to consecutive instructions unless altered by branch or jump instructions.*
4. *Control Flow Management:*
 - *In conditional branching or loops, the PC is updated with specific addresses, enabling control flow changes (e.g., jumps, function calls).*

Role of Stack Pointer (SP)

The Stack Pointer (SP) is another special-purpose register in the CPU that:

1. *Points to the Top of the Stack:*
 - *The SP holds the memory address of the top element of the stack (last-in, first-out structure).*
2. *Used in Function Calls and Interrupts:*
 - *During a function call, the return address and local variables are pushed onto the stack, and the SP is updated.*
 - *For interrupts, the CPU uses the SP to save the current state.*
3. *Manages Memory Allocation for the Stack:*
 - *When data is pushed (added) onto the stack, the SP is decremented (in a descending stack) or incremented (in an ascending stack).*
 - *When data is popped (removed), the SP is updated accordingly.*
4. *Ensures Program State Preservation:*
 - *The SP helps maintain a record of return addresses, function parameters, and local variables, which is crucial for multitasking and recursion.*

What is an Interrupt?

An interrupt is a signal generated by hardware or software to indicate that an event needs immediate attention from the CPU. Interrupts allow a system to respond to real-time events without continuously polling for their occurrence.

Types of Interrupts:

- 1. Hardware Interrupts:*

- Generated by external hardware devices (e.g., keyboard, mouse, disk, or network interface) to notify the CPU of an event like key presses or incoming data.*

- 2. Software Interrupts:*

- Triggered by software instructions to request specific CPU services (e.g., system calls).*

- 3. Internal Interrupts (Exceptions):*

- Caused by errors or exceptional conditions during program execution (e.g., divide-by-zero, invalid memory access).*
-